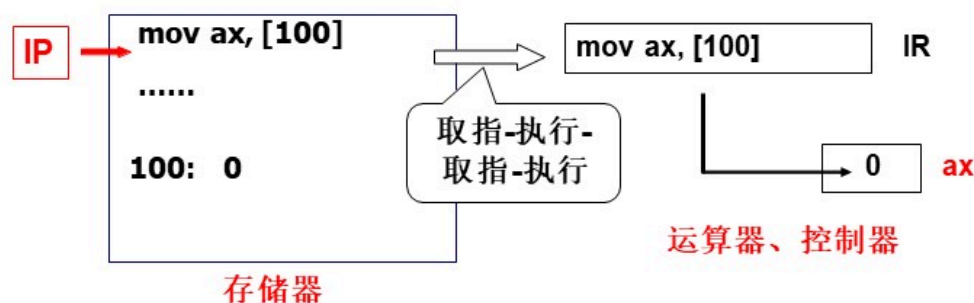


第4章 存储管理

- 4.1 内存使用与分段
- 4.2 内存分区与分页
- 4.3 多级页表与快表
- 4.4 段页结合的实际内存管理
- 4.5 内存换入-请求调页
- 4.6 内存换出

4.1 内存使用与分段



```
int main(int argc, char*  
argv[])  
{ int i, to, sum =  
  0; to =  
  atoi(argv[1]);  
  ...  
}
```



```
C:\>sum 12345  
76205685  
C:\>sum 32178  
517727931
```

如何让内存用起来?

仍然从计算机如何工作开始...

- 内存使用：将程序放到内存中，PC指向开始地址

那就让首先程序进入内存

```
int main(int argc, char*  
argv[])
```

```
{ ...  
.text  
_entry: //入口地址  
    call _main  
    call _exit  
_main:  
    ...  
    ret
```

```
_entry: //入口地址  
    call 40  
    call xx  
_main: //偏移是40  
    ...
```

内存地址

40

IP

→ 0

```
...  
_main: mov [300], 0  
...  
call xx  
call 40
```

物理内存

内存地址

1040

IP

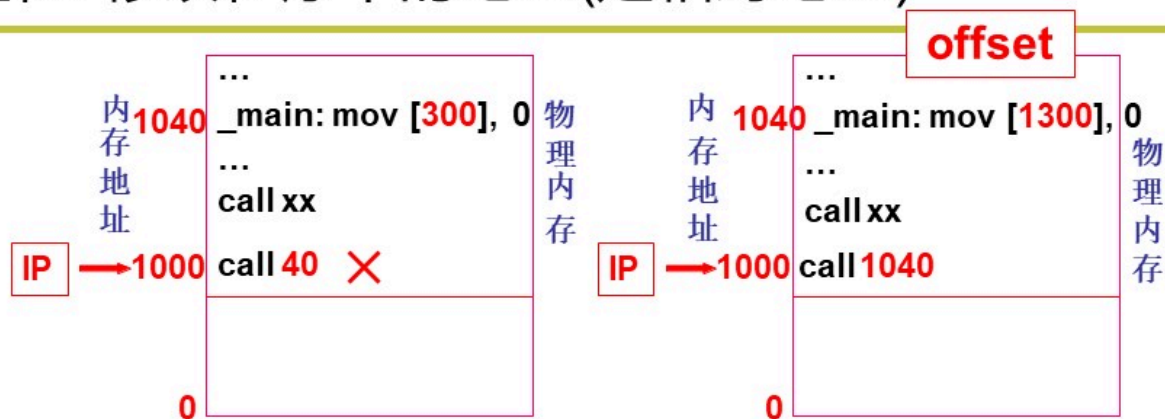
→ 1000

```
...  
_main: mov [300], 0  
...  
call xx  
call 40 ×  
  
0
```

物理内存

问题：现在内存是可以使用了，但是...

重定位: 修改程序中的地址(是相对地址)



■ 是什么时候完成重定位? 编译时 载入时

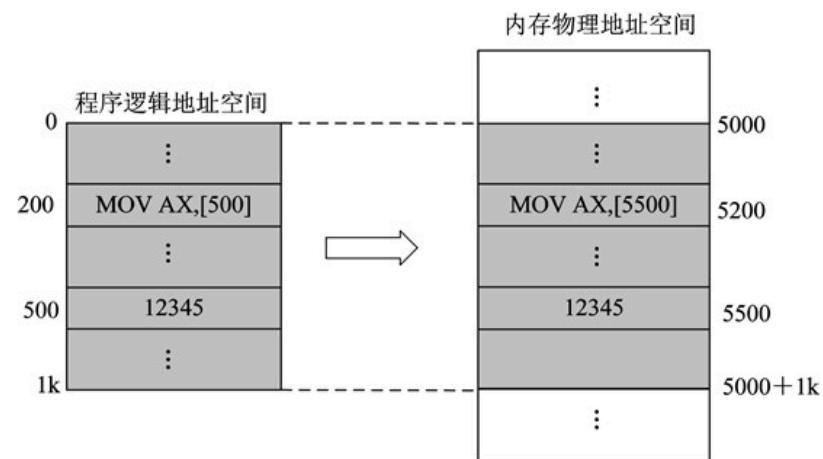
问题: 两种定义各自有什么特点?

- 编译时重定位的程序只能放在内存固定位置, 无需载入, 效率高
- 载入时重定位的程序, 找到空闲放入 (灵活), 但是一旦载入内存就不能动了。

• 程序装入

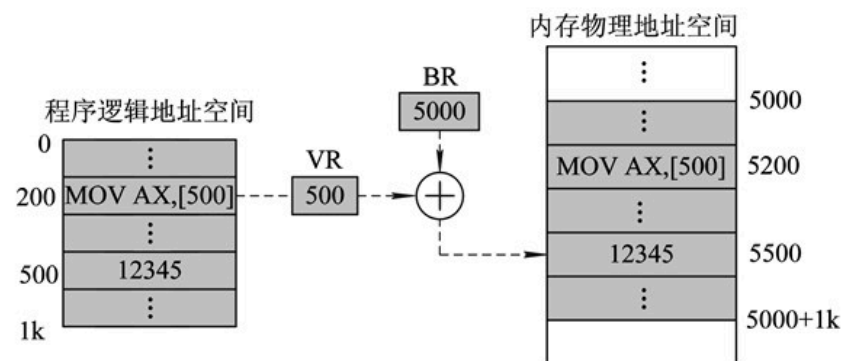
- 1. 程序装入
- 程序装入是指装入程序根据内存当前的实际使用情况，将装入模块（程序）装入到内存合适的物理位置。
- 要使装入内存的程序能够运行，必须将程序中的逻辑地址转换为机器能够直接寻址的物理地址。这种地址转换操作称为地址映射、地址转换或重定位。
- 程序装入分类：
 - 静态装入---在运行之前一次性地将程序（装入模块）装入内存，且在装入过程中同时完成逻辑地址（相对地址）到物理地址（绝对地址）的转换工作。
 - 运行时动态装入---把装入模块装入内存后，并不立即完成地址转换，而是把地址转换工作推迟到程序真正执行时才进行。

- 2. 静态重定位
- 指装入程序将程序（装入模块）装入到内存适当的位置后，在该程序运行之前一次性地将程序中所有指令中要访问的地址按下面的公式全部由相对地址（逻辑地址）转换为绝对地址（物理地址），并在程序运行过程中不再改变：
- 绝对地址 = 相对地址 + 程序存放的内存起始地址



- 静态重定位
- 优点:简单、容易实现, 不需要增加任何硬件设备, 可以通过软件全部实现。
- 缺点:
 - (1) 程序装入内存后, 在运行期间不允许该程序在内存中移动, 即无法实现内存重新分配, 因此内存的利用率不高。
 - (2) 如果内存提供的物理存储空间无法满足当前程序的存储容量, 则必须由用户在程序设计时采用某种方法来解决存储空间不足的问题, 这无疑增加了用户的负担。
 - (3) 不利于用户共享存放在内存中的同一个程序。如果几个用户要使用同一个程序, 就必须在各自的内存空间中存放该程序的副本, 这浪费了内存资源。

- 3. 动态重定位
- 指将装入模块装入内存后，并不立即完成相对地址到绝对地址的转换，地址的转换工作是在程序运行过程中进行的，即执行到要访问指令或数据的相对地址时再进行转换。
- 地址重定位机构需要一个（或多个）基地址寄存器（也称重定位寄存器，BR）和一个（或多个）程序逻辑地址寄存器（VR）。指令或数据在内存中的绝对地址与逻辑地址的关系为：
 - $\text{绝对地址} = (\text{BR}) + (\text{VR})$
- 动态重定位的过程如下：装入程序将程序（装入模块）装入到内存，然后将程序所装入的内存区域首地址作为基地址送入BR中。在程序运行过程中，当某条指令访问到一个相对地址时，则将该相对地址送入VR中；这时，硬件地址转换机构把BR和VR中的内容相加就形成了要访问的实际物理地址（绝对地址）



- 动态重定位的优点：
 - (1) 指令和数据的物理地址是在程序运行过程中由硬件动态形成的；
 - (2) 动态重定位的地址转换工作在程序真正执行到该指令时才进行；
 - (3) 动态重定位有利于对程序段进行共享。
- 缺点： (1) 需要硬件支持； (2) 实现存储管理的软件算法比较复杂。

单选题 1分

下面关于重定位的描述中，错误的是

- A 绝对地址是内存空间的地址编号**
- B 用户程序中使用从0地址开始的地址编号是逻辑地址**
- C 动态重定位中装入内存的程序仍保持原来的逻辑地址**
- D 静态重定位中装入内存的程序仍保持原来的逻辑地址**

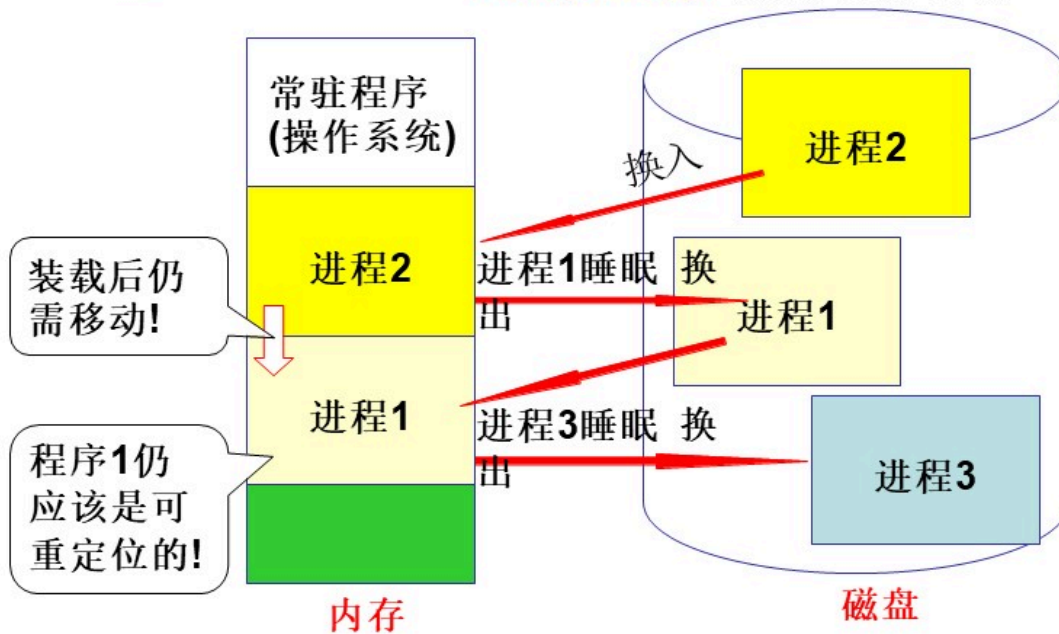
单选题 1分

采用动态重定位方式装入程序，其地址转换工作是在_____完成的

- ☐ A 程序装入时
- ☐ B 程序被选中时
- ☐ C 执行一条指令时
- ☐ D 程序在内存中移动时

程序载入后还需要移动...

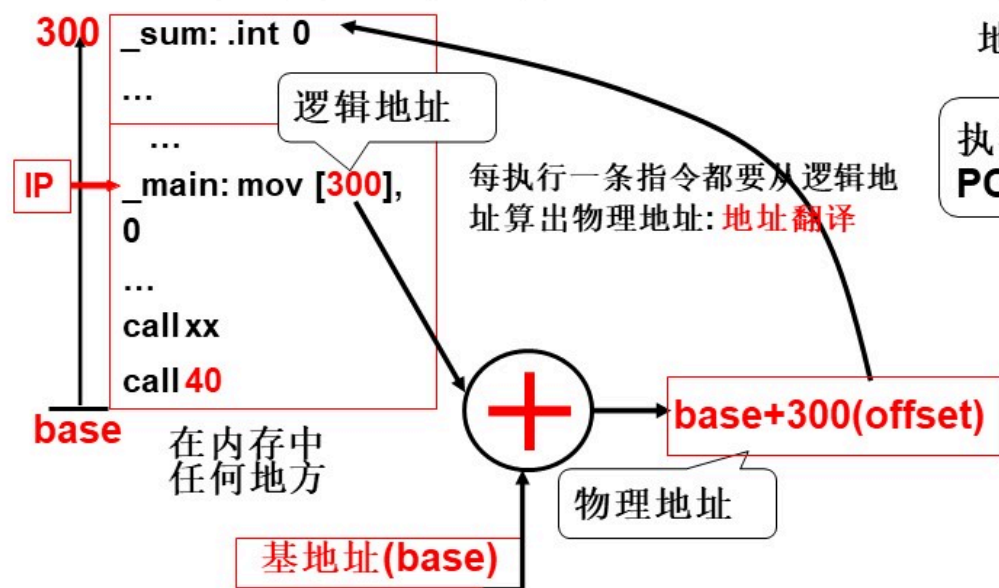
■ 一个重要概念: **交换(swap)** 充分利用内存



重定位最合适的时机 – 运行时重定位

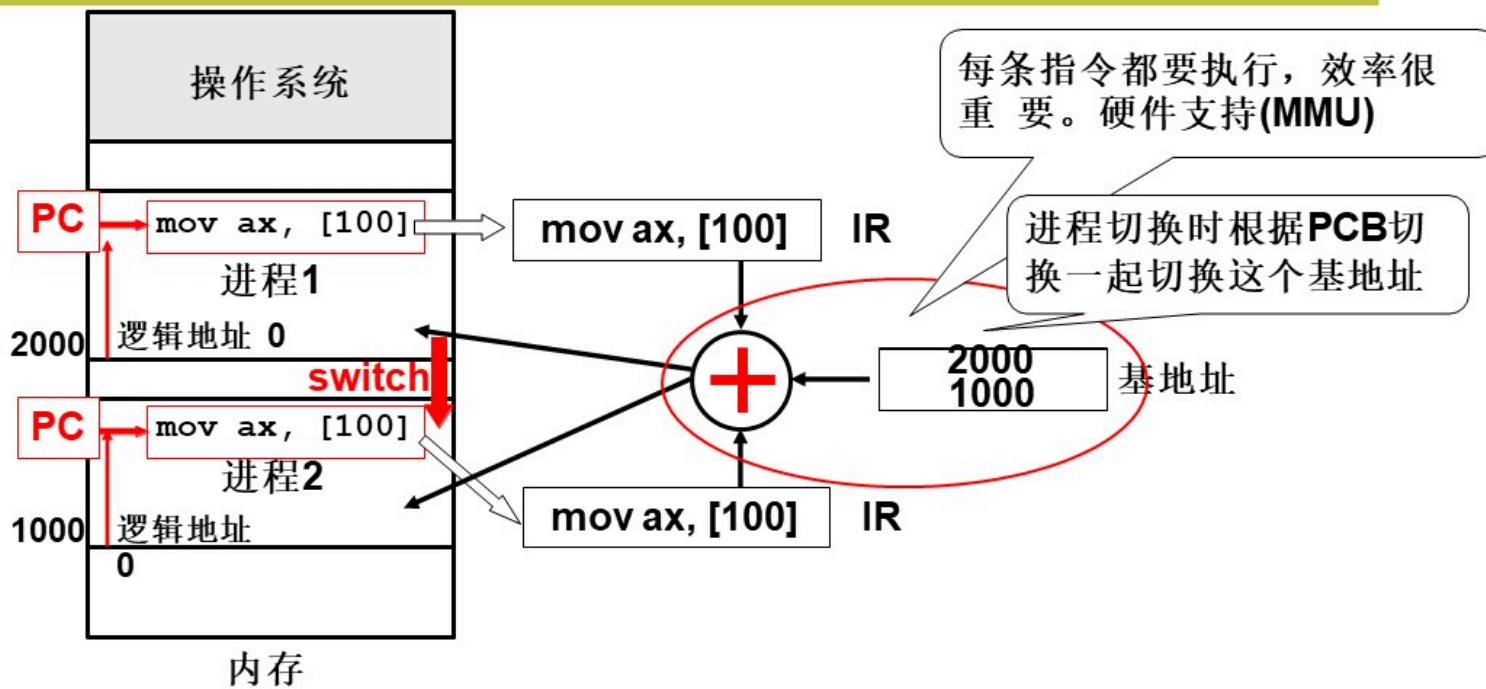
■ 在运行每条指令时才完成重定位

■ 每个进程有各自的基地址，放在哪里？ **PCB**



执行指令时第一步先从 **PCB** 中取出这个基地址

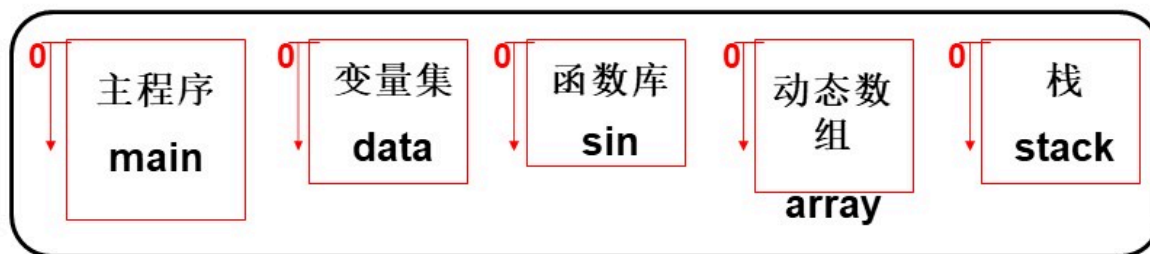
整理一下思路...



引入分段: 是将整个程序一起载入内存中吗?

程序员眼中的程序

- 由若干部分(段)组成, 每个段有各自的特点、用途: 代码段只读, 代码/数据段不会动态增长...

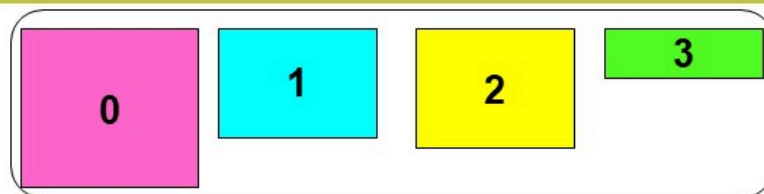


程序员眼中的一个程序

- 符合用户观点: 用户可独立考虑每个段(分治)
- 怎么定位具体指令(数据): <段号, 段内偏移>

如 `mov [es:bx], ax`

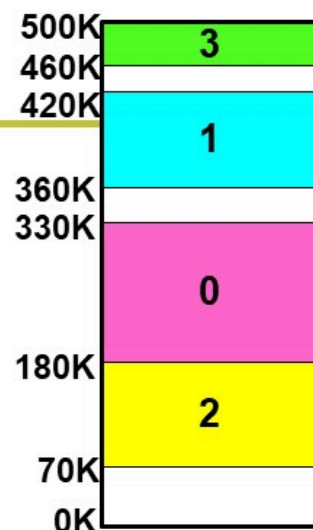
不是将整个程序，是将各段分别放入内存



■ 再进行运行时重定位会怎么样？

mov [DS:100], %eax jmp 100, CS

进程段表	段号	基址	长度	保护
	0	180K	150K	R
	1	360K	60K	R/W
	2	70K	110K	R/W



问题：假设 **DS=1**，**CS=0**，上面两条指令运行时重定位成什么？

那么 **jmp 500K** 呢？

单选题 1分

在分段存储管理中，CPU每次在内存中存取一次数据需要____次访问内存。

- ☐ A 1
- ☐ B 2
- ☐ C 3
- ☐ D 4

单选题 1分

一个分段存储管理系统中，地址长度为32位，其中段号占8位，则最大段长是_____。

- ☐ A 2^8 字节
- ☐ B 2^{16} 字节
- ☐ C 2^{24} 字节
- ☐ D 2^{32} 字节

单选题 1分

在分段存储管理方式中，_____。

- ☐ A 以段为单位分配内存，每段是一个连续存储区
- ☐ B 段与段之间必定不连续
- ☐ C 段与段之间必定连续
- ☐ D 每个段都是等长的

填空题 6分

在一个分段存储管理系统中，其段表如下左表所示，试求右表中逻辑地址所对应的物理地址 [填空1] [填空2] [填空3] [填空4] [填空5] [填空6] 。

段号	段长	内存起始地址
0	500	210
1	20	2350
2	90	100
3	590	1350
4	95	1938

段号	段内地址
0	430
1	10
2	500
3	400
4	112
5	32

4.2 内存分区与分页

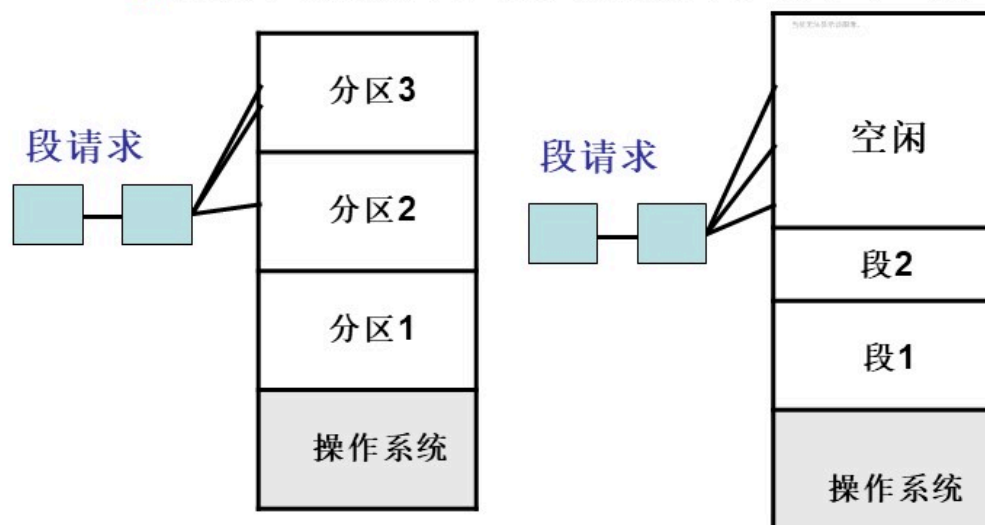
接下来的问题是内存怎么割？

这样就可以将程序的各个段载入到相应的内存分区中了

固定分区 与 可变分区

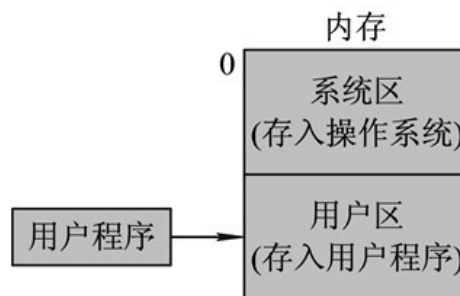
■ 给你一个面包，一堆孩子来吃，怎么办？

- 等分，操作系统初始化时将内存等分成 k 个分区
- 但孩子有大有小，段也有大有小，需求不一定



4.2 内存分区与分页

- 分区式存储管理——根据用户程序的需求为其在内存分配一段连续的存储空间。
- **单一连续分区存储管理**
 - 1. 实现原理
 - 只适合于单用户单任务操作系统，是一种最简单的存储管理方式。
 - 单一连续分区管理将内存空间划分为系统区和用户区两部分（如下图所示）。系统区仅供操作系统使用，通常放在内存的低地址部分，系统区以外的全部内存空间就是用户区，提供给用户使用。用户程序由装入程序从用户区的低地址开始装入且只能装入一个程序运行。用户区装入一个程序后，内存中剩余的区域则无法再利用。

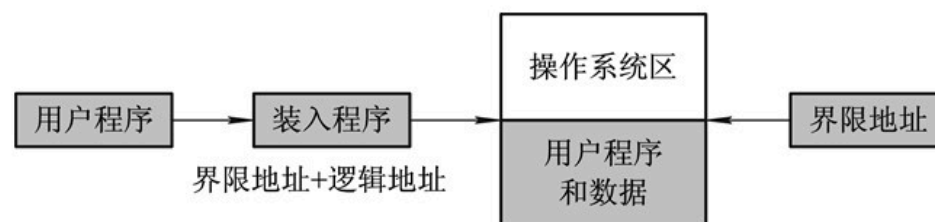


单一连续分区存储管理

- 2. 分配与回收
- 分配过程是先将待装入内存的程序与用户区进行大小比较，若程序所需内存空间没有超过用户区的大小，则为它分配内存空间；否则内存分配失败。回收操作则是在用户区的程序运行结束后，将该区域标志置为未分配即可。

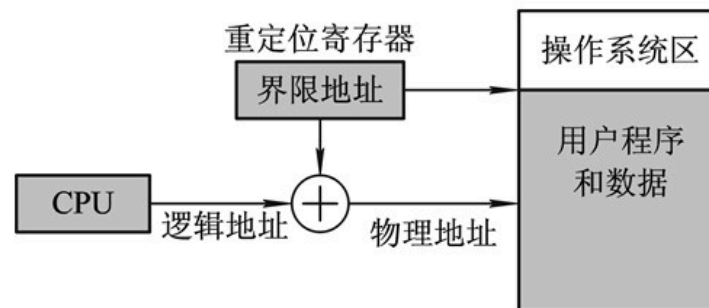
单一连续分区存储管理

- 3. 地址转换与存储保护
- (1) 采用静态重定位方式
- 单一连续分区存储管理的地址转换多采用静态重定位，即用户程序在装入内存时采用静态重定位一次性对所有数据和指令中的逻辑地址进行转换。程序执行期间，不允许指令和数据再改变地址，也不允许程序在内存中移动位置。



单一连续分区存储管理

- (2) 采用动态重定位方式
- 设置一个重定位寄存器并用它来指明内存中系统区和用户区的地址界限，同时又作为用户区的基地址。用户程序由装入程序装入到从界限地址开始的内存区域，但这时并不进行地址转换。地址转换要推迟到程序执行过程中执行到某指令中或某个数据的逻辑地址时，这时就动态地将这个逻辑地址与重定位寄存器中的值相加得到要访问的物理地址



单一连续分区存储管理

- 4. 单一连续分区管理的优缺点
- 优点：
 - ① 管理简单，开销小；
 - ② 安全性高，除了系统区外，用户区中只有一个程序，不存在多个程序相互影响的问题；
 - ③ 采用静态重定位方式时不需要硬件支持。
- 缺点：
 - ① 不支持多用户；
 - ② 程序的地址空间受用户区空间大小的限制，这是因为程序在运行前必须一次性装入内存的连续区域，若程序的地址空间比用户区大则无法装入；
 - ③ 由于一个程序独占系统资源，这样会造成系统资源的严重浪费。

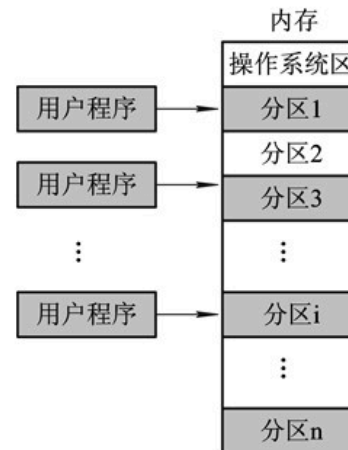
4.2 内存分区与分页

固定分区存储管理

1. 实现原理

将内存系统区之外的用户空间划分成若干个固定大小的区域，每个区域称为一个分区并可装入一个用户程序运行。分区一旦划分完成，就在系统的整个运行期间保持不变。

每个分区允许装入一道程序运行，则系统允许在内存中同时装入多道程序并发执行。



固定分区存储管理

2. 分区划分

在固定分区存储管理方式中，分区的数目和每个分区的大小由系统操作员或操作系统决定。

分区划分方式：

(1) 分区大小相等。优点：管理简单，缺点：缺乏灵活性。

例如，若程序过小则会造成内存空间浪费，若程序过大则因其无法装入分区而导致不能运行。

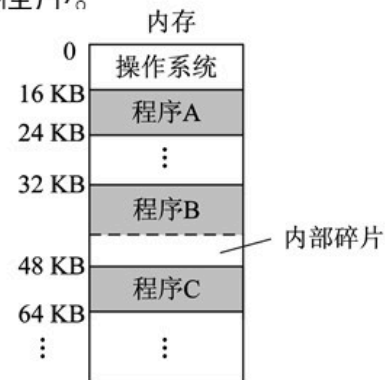
(2) 分区大小不等。适量的中分区和较少的大分区。装入程序可以根据用户程序的大小将它装入到适当的分区。

固定分区存储管理

- 3. 内存空间的分配与回收
- (1) 数据结构
- 为了有效管理内存中各分区的分配与使用，系统建立了一张内存分区分配表，用来记录内存中所划分的分区以及各分区的使用情况。
- 分区分配表的内容包括分区号、起始地址、大小和状态；状态栏的值为0时表示该分区空闲可以装入程序。

分区号	大小(kb)	起始地址(kb)	状态
1	8	16	程序A
2	8	24	0
3	16	32	程序B
4	16	48	程序C
5	32	64	0
6	64	96	0

(a) 分区分配表



(b) 内存空间分配示意

固定分区存储管理

- (2) 内存空间分配
- 系统启动后在为程序分配分区之前，根据内存分区的划分在分区分配表中填入每个分区的起始地址和大小，并且将所有的状态栏均填入0表示这些分区可用。
- (3) 内存空间回收
- 当程序运行结束时，根据程序名检索分区分配表，从状态栏信息可找到该程序所使用的分区，然后将该分区状态栏置为0，表示该分区已经空闲可以装入新的程序。
- 4. 地址转换与存储保护
- 固定分区存储管理的地址转换可以采用静态重定位方式：
- $\text{物理地址} = \text{逻辑地址} + \text{分区起始地址}$
- 存储保护的方法是：在程序运行过程中，每当CPU获得物理地址时首先与上、下界寄存器的值进行比较，若超出上、下界寄存器的值就发出地址越界中断信号，并由相应的中断处理程序处理。

固定分区存储管理

- 5. 固定分区分配的优缺点
- 优点：通过分区分配表来实现内存分配与回收，而且程序执行时采用静态重定位这种方式简单易行，CPU利用率较高。
- 缺点：程序大小受分区大小的限制，当分区较大而程序较小时容易形成内部碎片（一个分区内部浪费的空间称为内部碎片）而造成内存空间的浪费；
- 由于固定分区分配方式使分区总数固定，也就限制了并发执行程序的数量。

4.2 内存分区与分页

• 可变分区存储管理

- 1. 实现原理
- 在程序装入内存之前并不建立分区，内存分区是在程序运行时根据程序对内存空间的需要而动态建立的。
- 分区的划分时间、大小及其位置都是动态的。
- 2. 数据结构
- 包括：已分配分区表、空闲分区表及空闲分区链。
- **已分配分区表**用于登记内存空间中已经分配的分区，每个表项记录一个已分配分区，其内容包括分区号、起始地址、大小和状态。
- **空闲分区表**则记录内存中所有空闲的分区，每个表项记录一个空闲分区，其内容包括分区号、起始地址、大小和状态。
- **空闲分区链**以系统当前的空闲分区为结点，利用链指针将所有空闲分区结点链接成一个双向循环队列。

可变分区的管理过程 — 核心数据结构



空闲分区表

始址	长度
250K	250K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	Seg1
200K	50K	Seg2

可变分区的管理—请求分配



■ 段内存请求: reqSize = 100K

空闲分区表

始址	长度
250K	250K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	Seg1
200K	50K	Seg2
250K	100K	Seg3

可变分区的管理—释放内存



■ 段2不再需要，释放内存

空闲分区表

始址	长度
350K	150K
200K	50K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	Seg1
200K		Seg2
250K	100K	Seg3

可变分区的管理—再次申请

■ 又一个段提出内存请求: reqSize=40K,
怎么办?

■ 有2个空闲分区, 选哪一个?

■ 首先适配: (200,50)

■ 最佳适配: (200,50)

■ 最差适配: (350,150)

空闲分区表

始址	长度
350K	150K
200K	50K



可变分区存储管理

- 3. 分配算法
 - (1) 首次适应算法 (首先匹配)
 - 要求空闲分区按内存地址递增的次序排列，每当一个程序申请装入内存时，管理程序在空闲分区链上按内存地址递增的顺序从链首开始查找空闲分区，直到找到一个最先满足此程序要求的空闲分区，并按此程序的大小从该空闲分区中划出一块连续的内存区域给其使用。
 - 特点：每次都从内存的低地址部分开始查找满足要求的空闲分区。
 - (2) 最佳适应算法 (最佳匹配)
 - 要求空闲分区按分区大小递增的次序排列在空闲分区链上，当为用户程序分配内存空闲分区时，则从空闲分区链链首的最小空闲分区开始查找，找到的第一个大小满足程序要求的空闲分区就是最佳空闲分区；该分区能满足程序的内存要求，并且在分配后剩余的空闲空间最小。
 - 优点：保留了内存中的大空闲区，当大程序到来时有足够大的空闲区可以为其分配。

可变分区存储管理

- (3) 最差适应算法 (最差匹配)
 - 要求空闲分区按分区大小递减的次序排列在空闲分区链上，每一次总是把空闲分区链链首的最大的空闲分区分配给请求的用户程序；若该空闲分区小于程序要求的大小则分配失败；若能够满足程序的要求，则按程序要求的大小从该空闲分区中划出一块连续区域分配给它。
 - 优点：有利于后期再分配。

单选题 1分

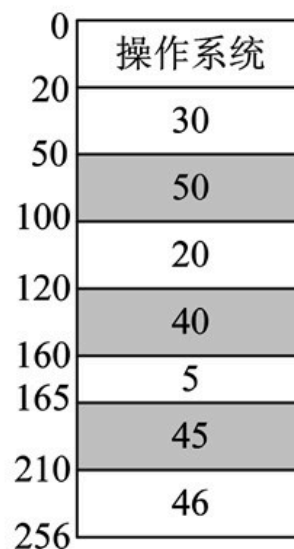
如果某操作系统中的段内存请求很不规则，有时候需要很大的一个内存块，有时候又很小，此时用哪种分区分配算法最好？

- ☐ A 最先适配
- ☐ B 最佳适配
- ☐ C 最差适配
- ☐ D 没有区别

单选题 1分

有一程序序列：程序A要求18KB，程序B要求25KB，程序C要求30KB，初始内存分配情况如图所示（其中阴影为已分配区）。问首次适应算法、最佳适应算法和最差适应算法中哪种能满足该程序序列的分配？

- ☐ A 最先适配
- ☐ B 最佳适配
- ☐ C 最差适配
- ☐ D 没有区别



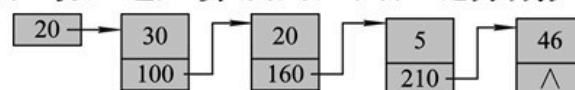
可变分区存储管理

- 例4.1 有一程序序列：程序A要求18KB，程序B要求25KB，程序C要求30KB，初始内存分配情况如图所示（其中阴影为已分配区）。问首次适应算法、最佳适应算法和最差适应算法中哪种能满足该程序序列的分配？

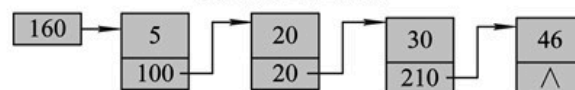
0	操作系统
20	30
50	50
100	20
120	40
160	5
165	45
210	46
256	

可变分区存储管理

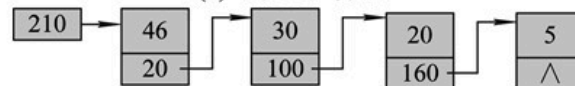
【解答】 结合系统中初始内存的分配情况，建立的首次适应算法、最佳适应算法和最差适应算法的空闲区链分别如图：



(a) 首次适应算法



(b) 最佳适应算法



(c) 最差适应算法

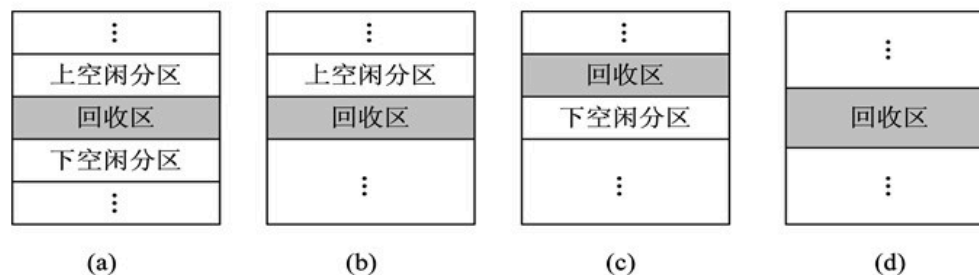
对于首次适应算法，程序A分配30KB的空闲分区，程序B分配46KB的空闲分区，此后就无法为程序C分配合适的空闲分区了。

对于最佳适应算法，程序A分配20KB的空闲分区，程序B分配30KB的空闲分区，程序C分配46KB的空闲分区。

对于最差适应算法，程序A分配46KB的空闲分区，程序B分配30KB的空闲分区，此后就无法为程序C分配合适的空闲分区了。

因此，本题最佳适应算法对这个程序序列的内存分配是合适的。答案为B

- 4. 分配与回收
- 系统进行内存分配时采用相应的分配算法来查找系统中是否存在满足要求的空闲分区；若找到就为程序分配相应的内存空间，并修改相关的数据结构，否则此次分配失败。
- 内存的回收指在程序运行结束后，系统要把该程序释放的内存空间及时收回以便重新分配给需要的程序。
- 回收的内存区与内存中的空闲分区在位置上存在4种关系：
 - (1) 回收的内存区与上、下两个空闲分区相邻
 - (2) 回收的内存区只与上空闲区相邻
 - (3) 回收的内存区只与下空闲分区相邻
 - (4) 回收的内存区上、下都不与空闲分区相邻



- 5. 地址转换与存储保护
 - (1) 地址转换
 - 步骤如下：
 - ① 当程序占用CPU时，进程调度程序就把该程序所占分区的起始地址送入基址寄存器，把程序所占分区的大小送入限长寄存器。
 - ② 程序执行过程中，每当CPU执行到涉及地址的指令时都要由地址转换机构把程序中的逻辑地址转换成物理地址。
 - (2) 存储保护
 - 基址寄存器和限长寄存器分别记录当前运行程序在内存中所占分区的起始地址和结束地址。当CPU执行到程序中涉及地址的指令时，必须核对转换后的物理地址是否满足条件“起始地址 $\leq$ 物理地址 $\leq$ 结束地址”，若满足则执行该指令，否则产生地址越界错误停止该程序的执行。

- 6. 碎片问题

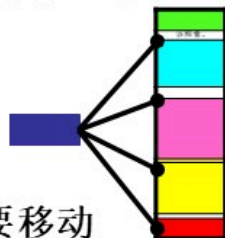
- 在采用可变分区存储管理方式下，随着分配与回收的不断进行，内存中会出现很多离散分布且容量很小的空闲小分区，虽然这些空闲小分区的总容量能够满足程序对内存的要求，但由于一个程序需要装入到一个连续的内存分区，而这些空闲小分区单个又不能满足程序对内存大小的需求，于是这些小的空闲分区就成为内存中无法再利用的资源，称为内存碎片或零头。
- 在可变分区分配中，系统解决外部碎片的思路是通过某种方法，将内存中无法利用的小空闲分区合并在一起组成一个较大的空闲分区来满足程序的需要，这种方法被称为紧凑技术。

可变分区造成的问题

■ 发起请求reqSize=160K怎么办？

- 总空闲空间>160，但没有一个空闲分区>160，怎么办？

- 这就是内存碎片



空闲分区表

始址	长度
350K	150K
200K	50K

- 将空闲分区合并，需要移动1个段(复制内容)：内存紧缩
- 内存紧缩需要花费大量时间，如果复制速度1M/1秒，则1G内存的紧缩时间为1000秒≈17分钟



从连续到离散...

■ 让给面包没有谁都不想要的碎末

- 将面包切成片，将内存分成页
- 针对每个段内存请求，系统一页一页的分配给给这个段

问题：此时需要内存紧缩吗？最大的内存浪费是多少？

页框7	
页框6	段0: 页3
页框5	段0: 页0
页框4	
页框3	段0: 页2
页框2	
页框1	段0: 页1
页框0	

页已经载入了内存，接下来的事情...

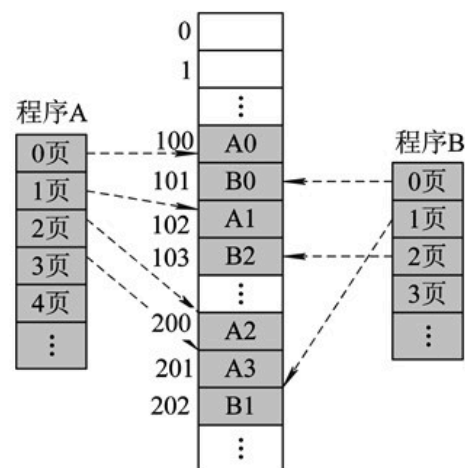
■ 页0放在页框5中，页0中的地址就需要重定位
页中的仍然是逻辑地址



页框7	
页框6	段0: 页3
页框5	段0: 页0
页框4	
页框3	段0: 页2
页框2	
页框1	段0: 页1
页框0	

• 分页存储管理的基本原理

- 1. 实现原理
- 在分页存储管理方式中，一个程序的逻辑地址空间被划分成若干个大小的区域，每个区域称为页或页面，并对程序所有的页从0开始依次编号。
- 在为程序分配内存时，允许以页为单位将程序的各个页分别装入内存中不相邻的物理块中。



分页存储管理的基本原理

- 2. 逻辑地址结构
- 程序的一维逻辑空间经过系统硬件自动分页后，形成“页号+页内地址”的地址结构
- 其中，0 ~ 11位是页内地址，即每个页的大小是4KB；12 ~ 31位是页号，即地址空间最多允许有1M个页。一维逻辑地址与页号和页内地址的关系是：
 - 一维逻辑地址 = 页号 × 页长 + 页内地址

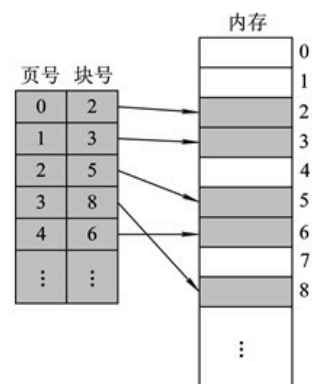


分页存储管理的基本原理

•3. 数据结构

• (1) 页表

•用来映射页号与内存物理块号之间的对应关系。最简单的页表由页号和对应的物理块号组成。



(2) 内存分配表

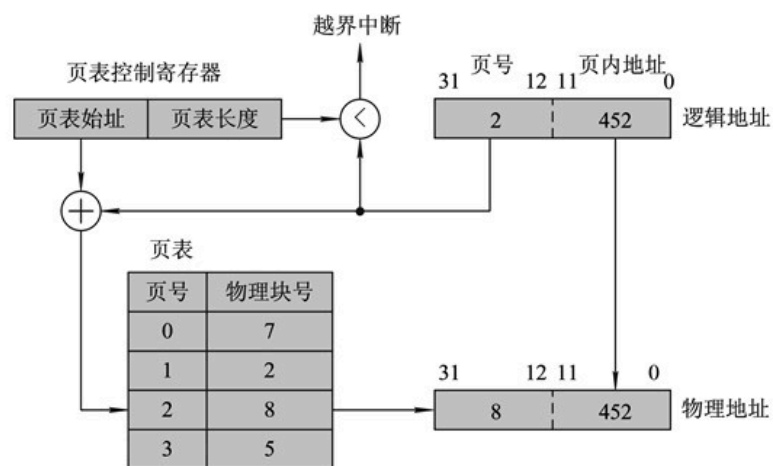
系统建立一张内存分配表来记录内存中物理块的分配情况。

最简单的办法是用一张位示图 (Bitmap) 来构成内存分配表。

	0	1	2	3	4	...	30	31
0	1	1	1	1	1	...	1	1
1	1	0	0	1	1	...	1	0
2	1	1	1	0	0	...	0	0
⋮	⋮	⋮	⋮	⋮	⋮	...	⋮	⋮
n	空闲块数							

• 分页存储管理的地址转换与存储保护

- 1. 基本地址转换
- 在分页存储管理系统中，系统为每个程序建立了一张页表。当程序被装入内存但尚未运行时，页表起始地址和页表长度等信息被保存到为该程序（进程）创建的PCB中；一旦调度程序调度该进程运行时，其PCB中保存的页表始址和页表长度信息便被装入页表控制寄存器中。



分页存储管理的地址转换与存储保护

- 地址转换机构按照下列步骤完成逻辑地址到物理地址的转换工作：
- (1) 地址转换机构自动将一维逻辑地址划分为页号和页内地址。
- (2) 将得到的页号与页表控制寄存器中存放的页表长度进行比较，如果页号大于页表长度，系统就产生地址越界中断，表示要访问的地址已经超过了该进程的地址空间；否则就根据页表控制寄存器中的页表始址找到页表在内存中存放的首地址。
- (3) 由页表的首地址（指向0号页）加上页号（页表项的相对位移）找到相应页号所在的表项位置，从而得到该页映射（装入）到内存中的物理块号。最后，将这个物理块号与页内地址拼接在一起（不是相加），即在逻辑地址中用物理块号取代页号就形成了要访问的物理地址。
- 实际要访问的物理地址与物理块号和页内地址的关系是：
- **物理地址 = 物理块号 × 页长 + 页内地址**

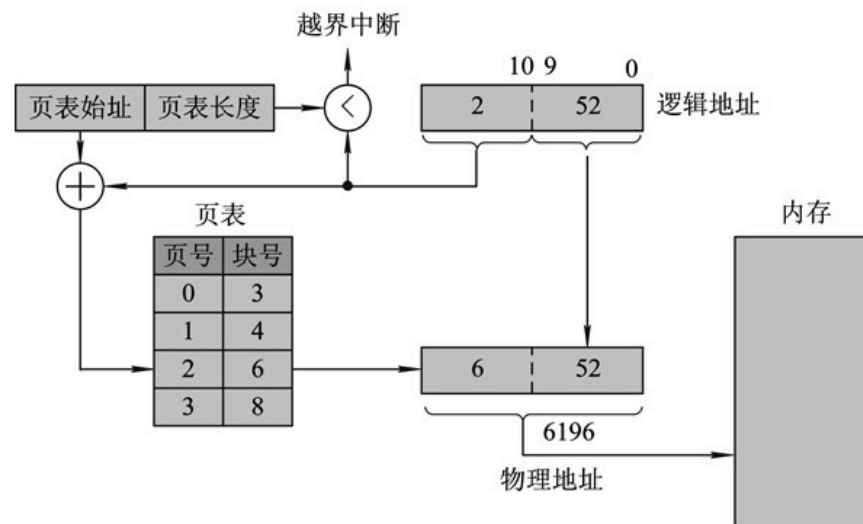
填空题 1分

在一采用分页存储管理的系统中，一程序有4个页并被分别装入到内存的第3、4、6、8号物理块中，已知页和物理块的大小均为1024B，该程序在CPU上运行并执行到如下的一条传送指令：

MOV AX,[2100]

请用地址转换图计算MOV指令中操作数2100的物理地址 **[填空1]**。

【解答】 本题的页的大小为1024B，可知页内地址为0~9位，即逻辑地址2100的页号=2100/1024=2（取整后），页内地址=2100-2×1024=52。逻辑地址2100的地址转换过程如图4-23所示。根据图4-23，可以得到页号2对应的物理块号为6，即物理地址=6×1024+52=6196



分页存储管理的地址转换与存储保护

•3. 页的共享和保护

• (1) 页的共享

•**数据共享**：允许不同的程序对共享的数据页使用不同的页号，只要让各自页表中有关表项指向共享信息的物理块即可；

•**程序共享**：必须把共享的程序安排到所有共享它的程序地址空间具有相同页号的页中。

• (2) 页的保护

•**逻辑地址转换成物理地址时的保护**：通过页号与页表长度的比较防止地址越界；

•**对共享信息的保护**：通常是在页表中增加一些标志位来设置存取控制字段，一般设置只读、读写、读和执行等权限；如果某进程试图去执行一个只允许读的内存物理块，系统就会发出访问性中断。

单选题 1分

当内存中所有碎片的容量之和大于某一程序所申请的内存容量时,_____。

- ☐ A 可以为这一程序直接分配内存
- ☐ B 不可以为这一程序分配内存
- ☐ C 拼接后可以为这一程序分配内存
- ☐ D 一定能够为这一程序分配内存

单选题 1分

分区管理和分页管理的主要区别是_____。

- ☐ A 分区管理中的空闲分区比分页管理中的页要小
- ☐ B 分页管理有地址映射（地址转换）而分区管理没有
- ☐ C 分页管理有存储保护而分区管理没有
- ☐ D 分区管理要求程序存放在连续的空间而分页管理没有这种要求

4.3 多级页表与快表

为了提高内存空间利用率，页应该小，但是页小了页表就大了...

页框7	
页框6	段0: 页3
页框5	段0: 页0
页框4	
页框3	段0: 页2
页框2	
页框1	段0: 页1
页框0	

页号	页框号	保护
0	5	R
1	1	R/W
2	3	R/W
3	6	R

页表会很大，页表放置就成了问题...

■ 纸上和实际使用总是存在很大差别!

- 页面尺寸通常为**4K**，而地址是**32位**的，有 **2^{20}** 个页面
- **2^{20}** 个页表项都得放在内存中，需要**4M**内存；系统中并发**10**个进程，就需要**40M**内存
- 实际上大部分逻辑地址根本不会用到

32位: 总空间[0, 4G]!

页号	页框号	保护	有效
0	5	R	1
1	1	R/W	1
2			0
3	6	R	1



页号	页框号	保护
0	5	R
1	1	R/W
3	3	R

第一种尝试，只存放用到的页

■ 很自然的想法：用到的逻辑页才有页表项

- 但页表中的页号不连续，就需要比较、查找，折半

$\log(2^{20})=20$ ，20次什么？

- 32位地址空间+4K页面+页号必须连续 $\Rightarrow 2^{20}$ 个页表项 $\Rightarrow$ 大页表占用内存，造成浪费

既要连续又要让页表占用内存少，怎么办

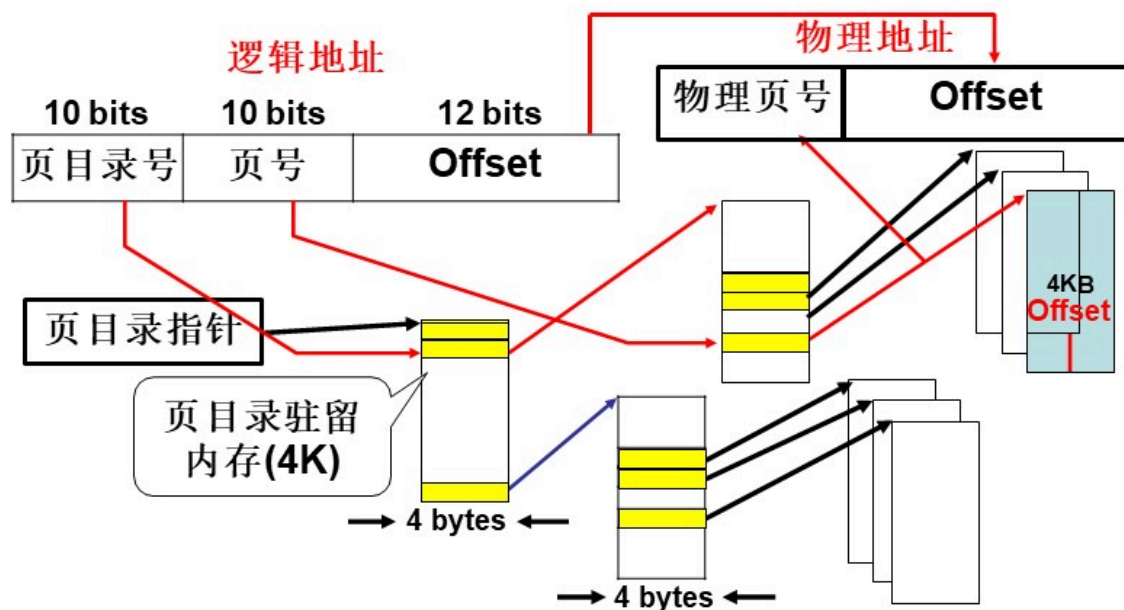
用书的章目录和节目录来类比思考...

页号	页框号	保护
0	5	R
1	1	R/W
3	3	R



页号	页框号	保护	有效
0	5	R	1
1	1	R/W	1
2			0
3	6	R	1

第二种尝试：多级页表，即页目录表（章）+页表（节）

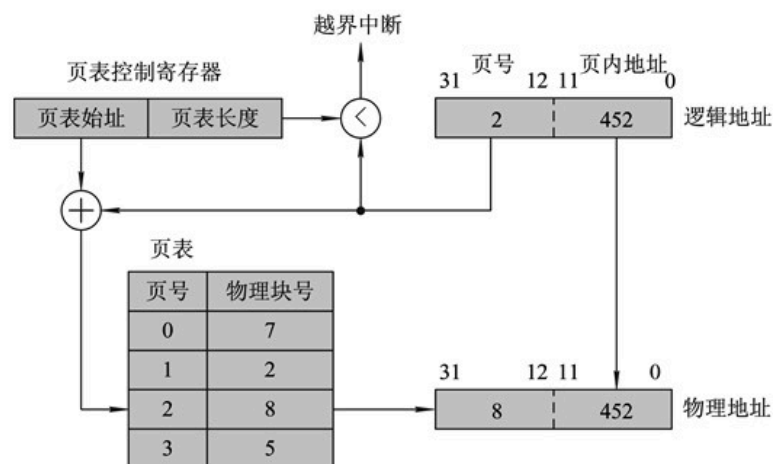


- 2^{10} 个目录项 $\times 4$ 字节地址 = 4K，总共需要 16K < 4M

单级页表 (2^{20} , 共4M)，两级页表，(3个4K再加上本身的4K, 共16K)

• 两级页表和多级页表

- 1. 基本地址转换
- 在分页存储管理系统中，系统为每个程序建立了一张页表。当程序被装入内存但尚未运行时，页表起始地址和页表长度等信息被保存到为该程序（进程）创建的PCB中；一旦调度程序调度该进程运行时，其PCB中保存的页表始址和页表长度信息便被装入页表控制寄存器中。



两级页表和多级页表

- 1. 两级页表

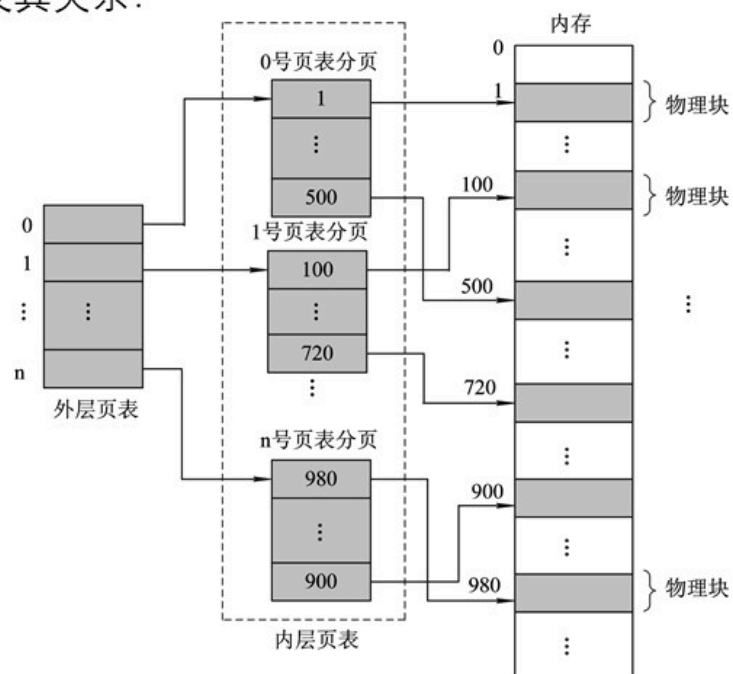
- 为了将一个大页表存放在内存中，可以考虑先对页表进行分页，然后把页表分页后形成的各个页（称为页表分页）分别存储在内存中不一定相邻的物理块中。由于各个页表分页以离散方式存放，为了将它们组织起来，可以以页表分页为单位建立**更高级索引**，该索引称为**外层页表**。每个页表分页在外层页表中有一个外层页表项，它记录了对应页表分页在内存中所存放的物理块号。

- 引入外层页表后，分页存储管理就建立在两级页表的基础上，这时进程的逻辑地址可以通过**外层页号**、**内层页号**、**页内地址**来描述。



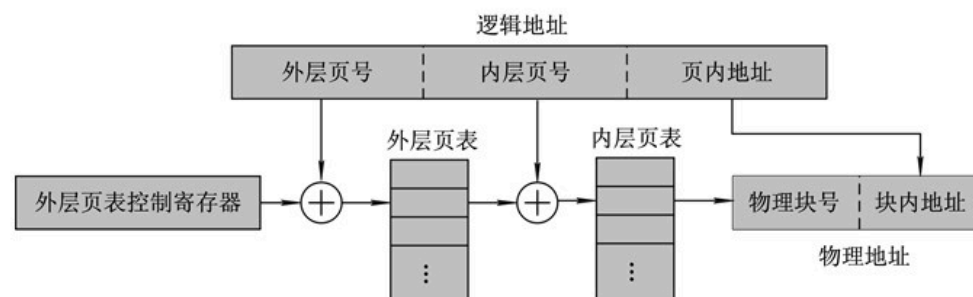
两级页表和多级页表

- 两级页表及其关系：



两级页表和多级页表

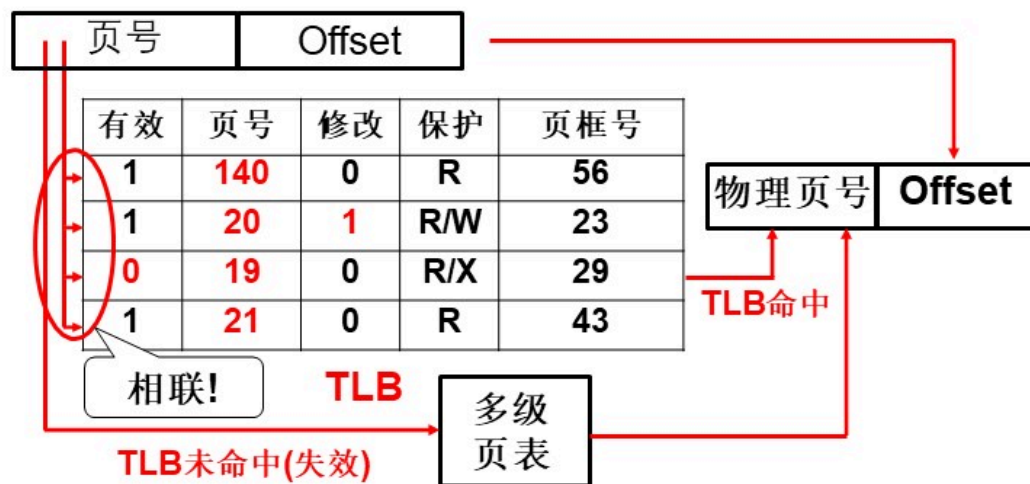
- 具有两级页表的地址转换过程：
 - 地址转换机构首先从外层页表控制寄存器中取出外层页表在内存中的起始地址，以进程逻辑地址中的外层页号检索外层页表，找到指定内层页表（页表分页）在内存中的起始地址；然后在内层页表中按内层页号找到指定的页表项，而该页表项则给出了要访问页在内存中的物理块号；最后，将该物理块号与逻辑地址中的页内地址进行拼接就形成了真正要访问的内存物理地址。



多级页表提高了空间效率，但在时间上？

多级页表增加了访存的次数，尤其是64位系统

逻辑地址



TLB是一组相联快速存储，是寄存器

TLB得以发挥作用的原因

- TLB命中时效率会很高，未命中时效率降低

$$\text{有效访问时间} = \text{HitR} \times (\text{TLB} + \text{MA}) + (1 - \text{HitR}) \times (\text{TLB} + 2\text{MA})$$

命中率!

内存访问时间!

TLB时间!

$$\text{有效访问时间} = 98\% \times (20\text{ns} + 100\text{ns}) + 2\% \times (20\text{ns} + 200\text{ns}) = 122\text{ns}$$

$$\text{有效访问时间} = 10\% \times (20\text{ns} + 100\text{ns}) + 90\% \times (20\text{ns} + 200\text{ns}) = 210\text{ns}$$

- 要想真正实现“近似访存1次”，
TLB的命中率应该很高
- TLB越大越好，但TLB很贵，通常
只有[64, 1024]

为什么TLB条目数可以在64-1024之间？

■ 相比 2^{20} 个页，64很小，为什么TLB就能起作用？

■ 程序的地址访问存在局部性

■ 空间局部性(Locality in Space)



程序多体现为循环、顺序结构

某内存引用模式



■ 计算机系统设计时应该充分利用这一局部性

单选题 1分

在多级页表中，每个页表项包含的地址转换信息是（）

- A 逻辑地址和物理地址的对应关系**
- B 物理页面的地址和页面大小**
- C 下一级页表的物理地址和页面大小**
- D 上一级页表的物理地址和页面大小**

单选题 1分

在使用快表进行地址转换时，如果发生了缺页中断，操作系统会执行什么操作？

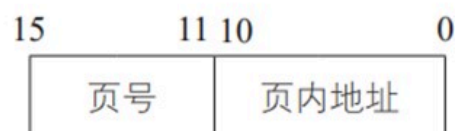
- A 将缺失的页面从磁盘读取到内存中**
- B 从快表中删除对应的页面映射**
- C 将进程切换到后台，等待缺页中断处理完毕后再恢复执行**
- D 重启计算机以重新初始化内存**

主观题 10分

【考研真题】某系统采用分页存储管理方式，拥有逻辑空间32 页，每页2KB；拥有物理空间1MB。

- (1) 写出逻辑地址的格式。
- (2) 若不考虑访问权限等，则进程的页表有多少项？每项至少有多少位？
- (3) 如果物理空间减少一半，则页表结构应相应地做怎样的改变？

(1) 该系统拥有逻辑空间32页，故逻辑地址中页号必须用5位来描述。而每页为2KB，因此，页内地址必须用 11位来描述，这样即可得到它的逻辑地址格式如下。



(2) 每个进程最多有32个页面，因此，进程的页表项最多为32项。若不考虑访问权限等，则页表项中只须给出页所对应的物理块块号，1MB的物理空间可分成 2^9 个内存块，故每个页表项至少有9位。

(3) 如果物理空间减少一半，则页表中的页表项数仍不变，但每项的长度可减少1位。

主观题 10分

已知某分页系统，内存容量为 64KB，页面大小为 1KB，对一个 4 页大的作业，其 0、1、2、3 页分别被分配到内存的 2、4、6、7 块中。

- (1) 将十进制的逻辑地址 1 023、2 500、3 500、4 500 转换为物理地址。
- (2) 以十进制的逻辑地址 1 023 为例，画出地址转换过程图。

(1) 对上述逻辑地址,可首先计算出它们的页号和页内地址(逻辑地址除以页面大小,得到的商为页号,余数为页内地址),然后通过页表将其转换成对应的物理地址。

① 逻辑地址1 023。 $\lfloor 1\ 023/1K \rfloor=0$, $1\ 023\%1K=1\ 023$,因此页号为0,页内地址为1 023,查页表找到对应的物理块号为2,故物理地址为 $2 \times 1K+1\ 023=3\ 071$ 。

② 逻辑地址2 500。 $\lfloor 2\ 500/1K \rfloor=2$, $2\ 500\%1K=452$,因此页号为2,页内地址为452,查页表找到对应的物理块号为6,故物理地址为 $6 \times 1K+452=6\ 596$ 。

③ 逻辑地址3 500。 $\lfloor 3\ 500/1K \rfloor=3$, $3\ 500\%1K=428$,因此页号为3,页内地址为428,查页表找到对应的物理块号为7,故物理地址为 $7 \times 1K+428=7\ 596$ 。

④ 逻辑地址4 500。 $\lfloor 4\ 500/1K \rfloor=4$, $4\ 500\%1K=404$,因此页号为4,页内地址为404,因页号大于页表长度,故产生越界中断。

(2) 逻辑地址1 023的地址转换过程如图1-5-1所示。

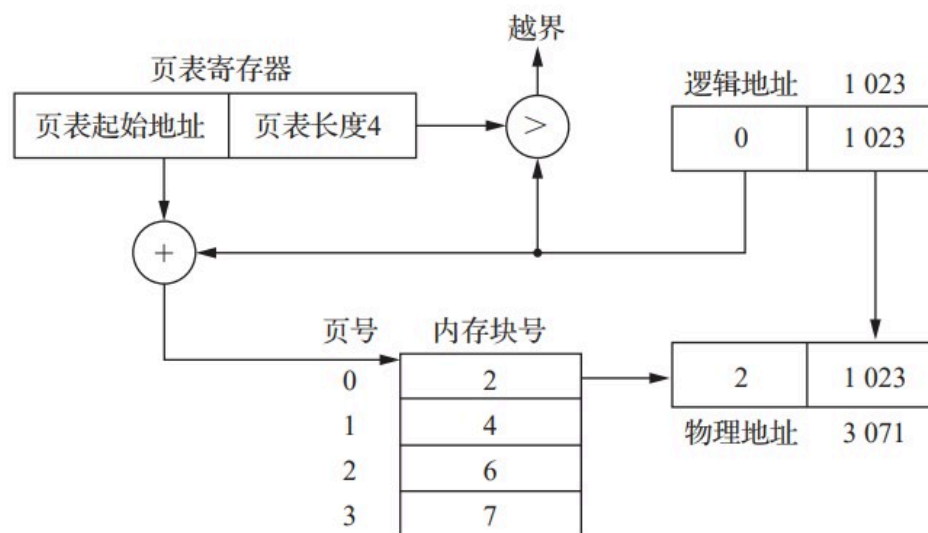


图 1-5-1 地址转换过程图

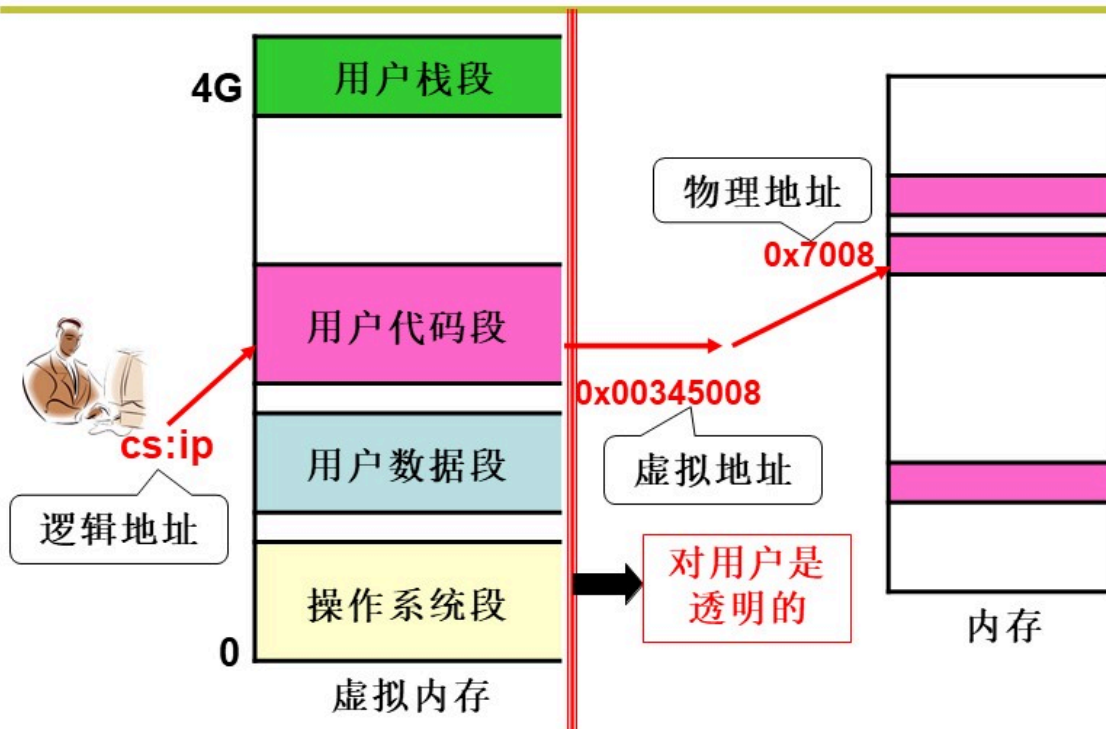
4.4 段页结合的实际内存管理

段、页结合：程序员希望用段，物理内存希望用页，
所以...

段、页同时存在：段面向用户/页面向硬件

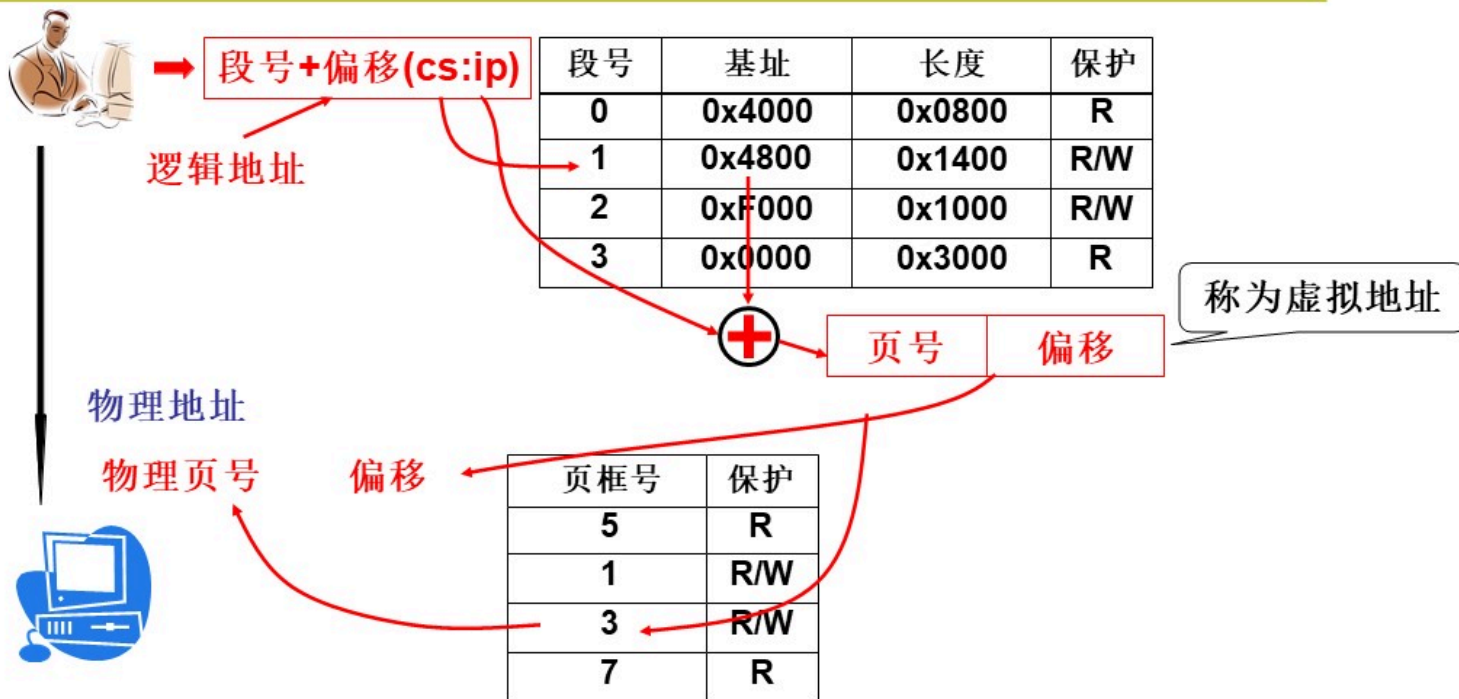
4.4 段页结合的实际内存管理

段、页同时存在：段面向用户/页面向硬件



问题：为什么称为
虚拟内存？

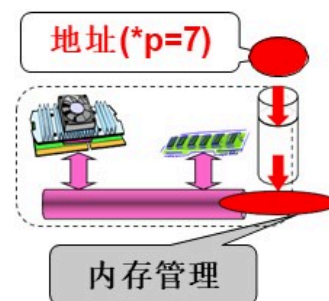
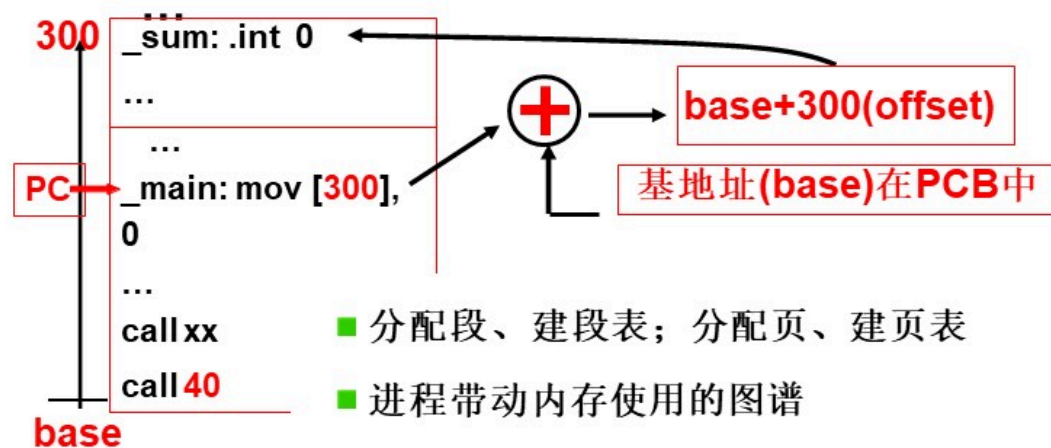
段、页同时存在是的重定位(地址翻译)



一个实际的段、页式内存管理

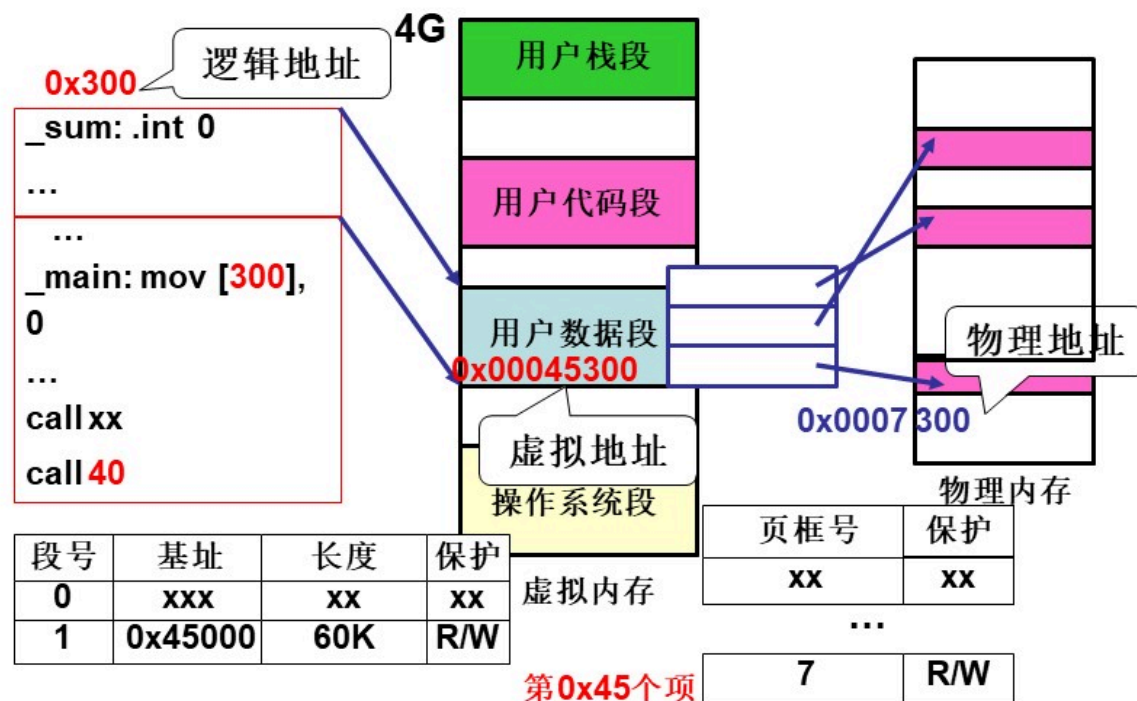
这个故事从哪里开始？

■ 内存管理核心就是内存分配，所以从程序放入内存、使用内存开始



- 分配段、建段表；分配页、建页表
- 进程带动内存使用的图谱
- 从进程**fork**中的内存分配开始..

段、页式内存下程序如何载入内存?



问题：整个过程分为哪几步？

单选题 1分

在段页式存储管理中，_____。

- ☐ A 每个作业或进程有一张段表和两张页表
- ☐ B 每个作业或进程的每个段有一张段表和一张页表
- ☒ C 每个作业或进程有一张段表并且每个段有一张页表
- ☐ D 每个作业或进程有一张页表并且每个段有一张段表

单选题 1分

在段页式内存管理中，页表和段表之间的映射关系是什么？

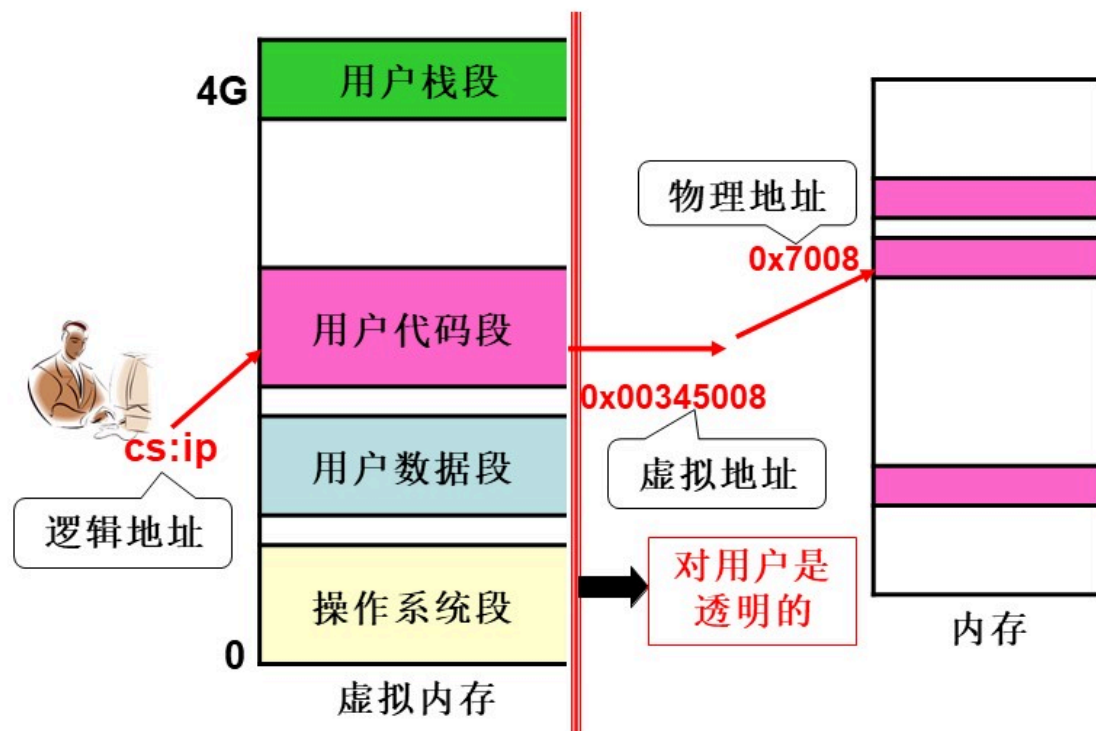
- ☐ A 每个页表条目映射到一个段。
- ☒ B 每个段表条目映射到一个页表。
- ☐ C 每个页表条目映射到一个物理地址。
- ☐ D 每个段表条目映射到一个物理地址。

单选题 1分

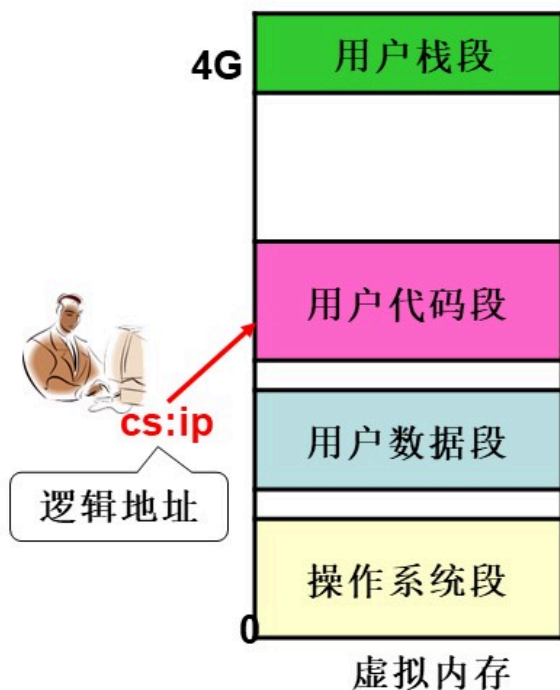
在段页式内存管理中，段表和页表都是用来干什么的？

- A 用于记录进程使用的虚拟地址空间和对应的物理地址**
- B 用于记录进程使用的物理地址空间和对应的虚拟地址**
- C 用于记录进程使用的文件和对应的内存块**
- D 用于记录进程使用的CPU寄存器和对应的内存地址**

4.5 内存换入-请求调页 段、页同时存在



用户眼里的内存!



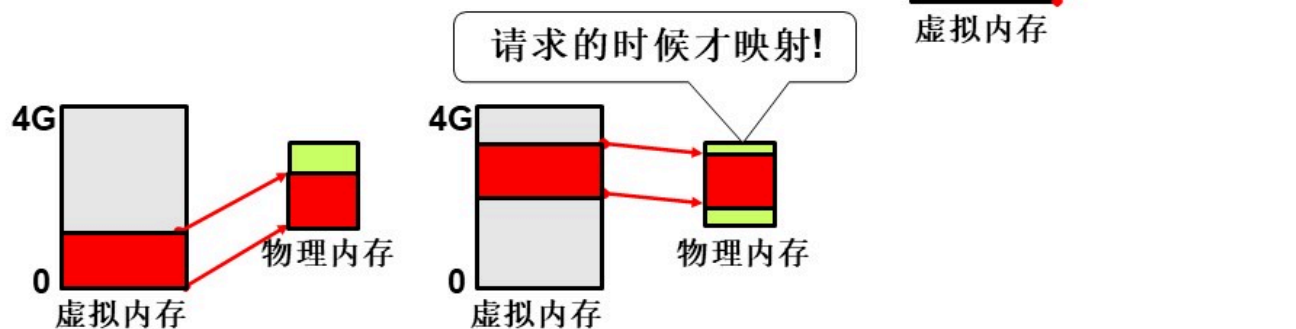
- **4GB**(大且规整)的“内存”，可供用户使用，如 `char *p, p=3G`，实际上就是用地址
- 用户可随意使用该“内存”，就象单独拥有**4G**内存
- 这个“内存”怎么映射到物理内存，用户全然不知

必须映射，否则
不能用!

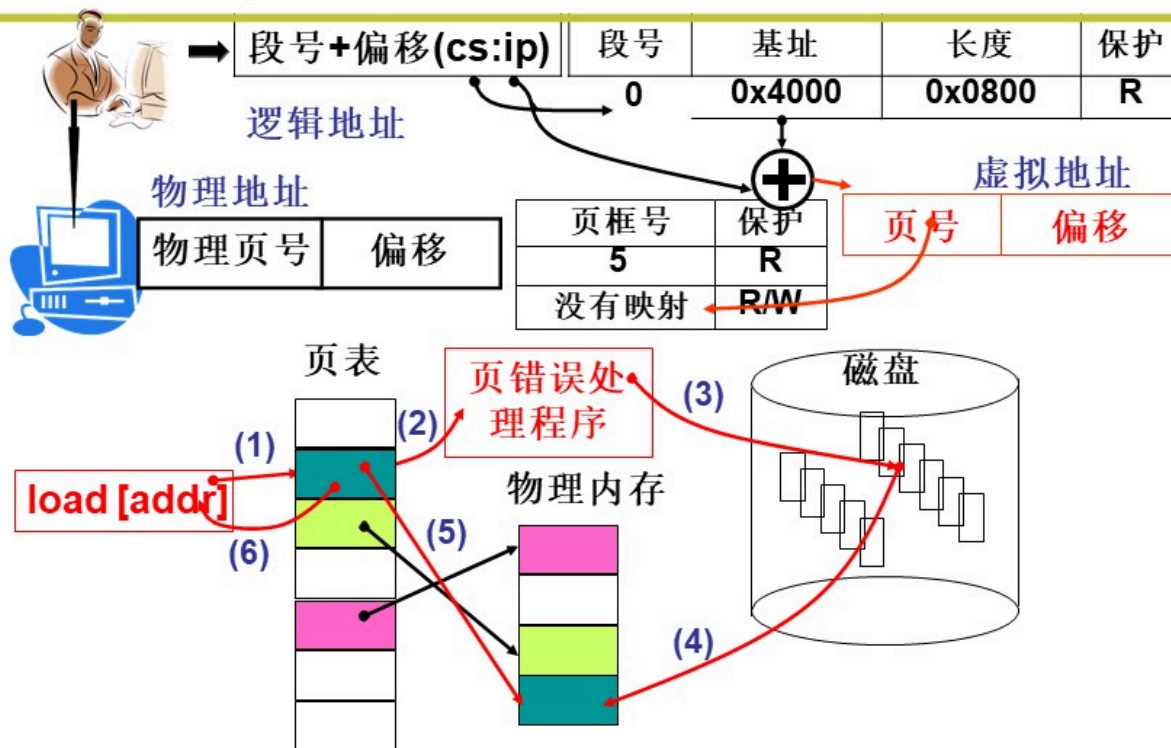
用换入、换出实现“大内存”

■ 左边4G，右边1G怎么办？

- 访问 $p(=0G - 1G)$ 时，将这部分映射到物理内存
- 再访问 $p(=3G - 4G)$ 时，再映射这一部分



请求调页



一个实际系统的请求调页

■ 这个故事从哪里开始？

■ 请求调页，当然从缺页中断开始

中断号	名称	说明
12	Segment not Present	描述符所指的段不存在
14	Page fault	页不在内存

```
void trap_init(void)
{ set_trap_gate(14, &page_fault); }
```

```
#define set_trap_gate(n, addr) \
    _set_gate(&idt[n], 15, 0, addr);
```

单选题 1分

在内存换入技术中，虚拟内存的作用不包括（ ）

- A 扩大物理内存的容量**
- B 缩小物理内存的容量**
- C 加快程序的运行速度**
- D 保证系统的稳定性和安全性**

单选题 1分

虚存管理和实存管理的主要区别是_____。

- ☐ A 虚存管理区分逻辑地址和物理地址，实存管理则不区分
- ☐ B 实存管理要求一程序在内存必须连续，而虚存管理则不需要连续的内存
- ☒ C 实存管理要求程序必须全部装入内存才开始运行，而虚存管理则允许程序在执行过程中逐步装入
- ☐ D 虚存管理以逻辑地址执行程序，而实存管理以物理地址执行程序

单选题 1分

为了使虚拟系统有效地发挥其预期的作用，所运行的程序应具有的特性是_____。

- ☐ A 该程序不应含有过多的I/O操作
- ☐ B 该程序的大小不应超过实际的内存容量
- ☐ C 该程序应具有较好的局部性
- ☐ D 该程序的指令相关不应过多

4.6 内存换出

get free page? 还是...

有换入，就应该有换出！

```
page=get_free_page();  
bread_page(page, current->executable->i_dev, nr);
```

■ 并不能总是获得新的页，内存是有限的

- 需要选择一页淘汰，换出到磁盘，选择哪一页？
- **FIFO**，最容易想到，但如果刚换出的页马上又要换入怎么办？
- 有没有最优的淘汰方法？**MIN**
- 最优淘汰方法能不能实现，是否需要近似？**LRU**

FIFO页面置换

- 一实例：分配了3个页框(frame)，页面引用序列为

A B C A B D A D B C B

D换A不太合适!

Ref: Page:	A	B	C	A	B	D	A	D	B	C	B
1	A					D				C	
2		B					A				
3			C						B		

- 评价准则：缺页次数；本实例，FIFO导致7次缺页

问题：换谁最合适？

MIN页面置换

■ **MIN (OPT)** 算法: 选最远将使用的页淘汰, 是最优方案

■ 继续上面的实例: (3frame)**A B C A B D A D B C B**

Ref: Page:	A	B	C	A	B	D	A	D	B	C	B
1	A									C	
2		B									
3			C			D					

■ 本实例, **MIN**导致5次缺页

■ 可惜, **MIN**需要知道将来发生的事... 怎么办?

LRU页面置换

■ 用过去的历史预测将来。**LRU**算法:选最近最长一段时间没有使用的页淘汰(最近最少使用)。

■ 继续上面的实例: (3frame)**AB C A B D A D B C B**

Ref: Page:	A	B	C	A	B	D	A	D	B	C	B
1	A									C	
2		B									
3			C			D					

■ 本实例, **LRU**也导致**5**次缺页

和**MIN**完全一样!

■ **LRU**是公认的很好的页置换算法, 怎么实现?

LRU的准确实现，用时间戳

■ 每页维护一个时间戳(**time stamp**)

■ 继续上面的实例: (3frame)**A B C A B D A D B C B**

	A	B	C	A	B	D	A	D	B	C	B
A	1	1	1	4	4	4	7	7	7	7	7
B	0	2	2	2	5	5	5	5	9	9	11
C	0	0	3	3	3	3	3	3	3	10	10
D	0	0	0	0	0	6	6	8	8	8	8

time
stamp

选具有最小时间戳的页!

选A淘汰!

■ 每次地址访问都需要修改时间戳，需维护一个全局时钟，需找到最小值...
实现代价较大

LRU准确实现，用页码栈

■ 维护一个页码栈

■ 继续上面的实例: (3frame)**A B C A B D A D B C B**

A	B	C	A	B	D	A	D	B	C	B
		C	A	B	D	A	D	B	C	B
	B	B	C	A	B	D	A	D	B	C
A	A	A	B	C	A	B	B	A	D	D

选栈底页淘汰!

页码栈

■ 每次地址访问都需要修改栈(修改10次左右栈指针)

... 实现代价仍然较大 ⇒ **LRU准确实现用的少**

LRU近似实现 – 将时间计数变为是和否

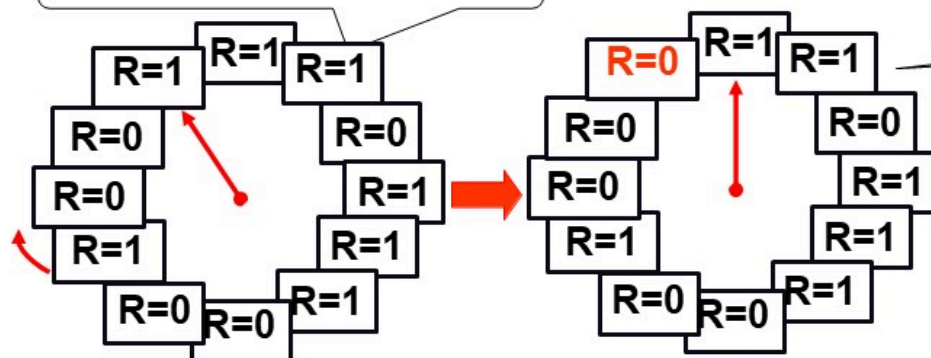
■ 每个页加一个引用位(reference bit)

■ 每次访问一页时，硬件自动设置该位为1

■ 选择淘汰页：扫描该位，是1时清0，并继续扫描；是0时淘汰该页

SCR这一实现方法称为
Clock Algorithm

组织成循环队列较合适！



再给一次机会(**Second
Chance Replacement**)

Clock算法的分析与改造

■ 如果缺页很少，会？ 所有的R=1

■ hand scan一圈后淘汰当前页，将调入页插入hand位置，hand前移一位

■ 原因：记录了太长的历史信息... 怎么办？

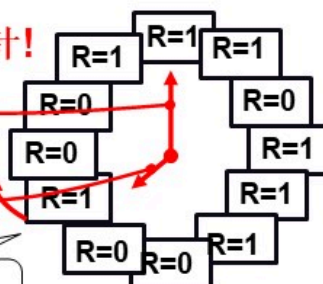
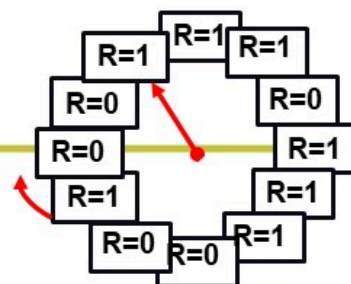
退化为FIFO!

■ 定时清除R位... 再来一个扫描指针!

用来清除R位，移动速度要快!

用来选择淘汰页，移动速度慢!

更像Clock吧!



单选题 1分

在请求分页存储管理中，若进程访问的页不在内存且内存又没有可用的物理块时，系统正确的处理顺序为_____。

- ☐ A 决定淘汰页，页调出，缺页中断，页调入
- ☐ B 决定淘汰页，页调入，缺页中断，页调出
- ☒ C 缺页中断，决定淘汰页，页调出，页调入
- ☐ D 缺页中断，决定淘汰页，页调入，页调出

单选题 1分

在页面置换算法中，FIFO算法有以下哪个缺点

- A** 它会选择一些常用的页面进行置换
- B** 它会选择一些最近被使用过的页面进行置换
- C** 它可能会导致页面抖动
- D** 它需要记录每个页面最近的使用情况

主观题 10分

有一个请求分页式虚拟存储器系统，分配给某进程 3 个物理块，开始时内存中预装入第 1,2,3 个页面，该进程的页面访问序列为 1, 2, 4, 2, 6, 2, 1, 5, 6, 1。

(1) 若采用 OPT (optimal, 最佳) 页面置换算法，则访问过程发生的缺页率为多少？

(2) 若采用 LRU 页面置换算法，则访问过程中的缺页率为多少？

(1) 采用OPT页面置换算法时，因为内存中预装入了前3个页面，所以前3个页面不发生缺页。访问过程中发生了2次缺页，缺页率 $f=2/10 \times 100\%=20\%$ 。页面置换走向如图1-6-1所示。

访问串	1	2	4	2	6	2	1	5	6	1
内存	1	1	1	1	1	1	1	1	1	1
		2	2	2	2	2	2	5	5	5
			4	4	6	6	6	6	6	6
缺页					√			√		

图 1-6-1 采用OPT页面置换算法时的页面置换走向

(2) 采用LRU页面置换算法时，访问过程中发生了4次缺页，缺页率 $f=4/10 \times 100\%=40\%$ 。页面置换走向如图1-6-2所示。

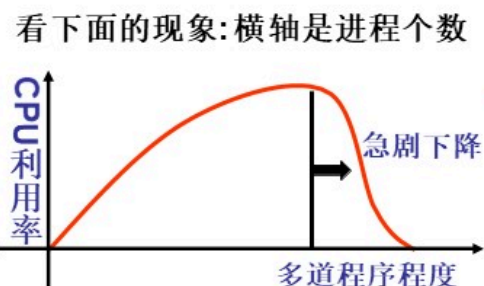
访问串	1	2	4	2	6	2	1	5	6	1
内存	1	1	1	1	6	6	6	5	5	5
		2	2	2	2	2	2	2	6	6
			4	4	4	4	1	1	1	1
缺页					√		√	√	√	

图 1-6-2 采用LRU页面置换算法时的页面置换走向

置换策略有了，还需要解决一个问题

■ 给进程分配多少页框(帧frame)

- 分配的多，请求调页导致的内存高效利用就没用了!
- 那分配的太少呢?



- 称这一现象为抖动(颠簸)(thrashing)

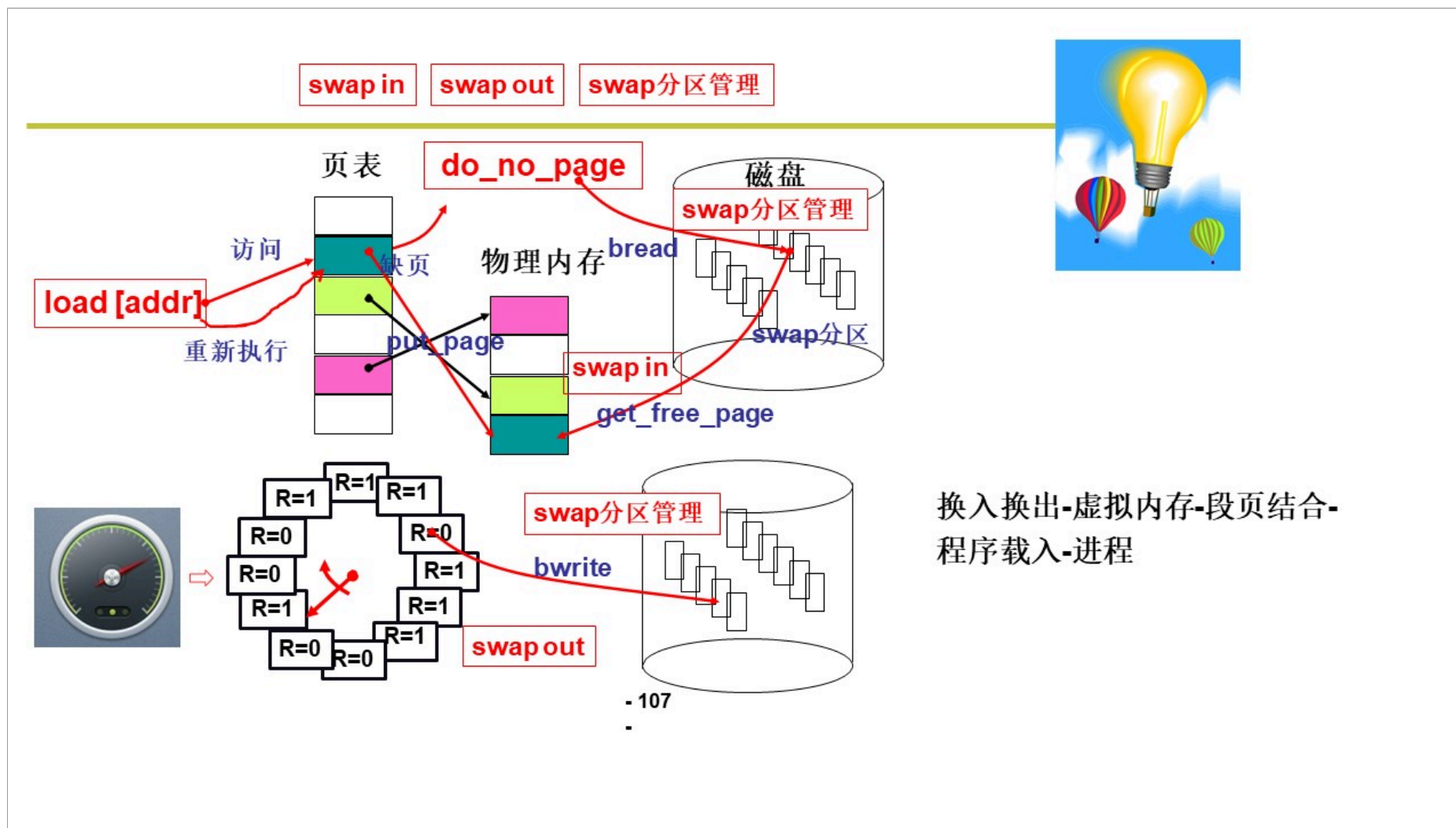
解释:系统内进程增多 $\Rightarrow$ 每个进程

的缺页率增大 $\Rightarrow$ 缺页率增大到一定程度, 进程总等待调页完成 $\Rightarrow$

CPU利用率降低 $\Rightarrow$ 进程进一步增多, 缺页率更大

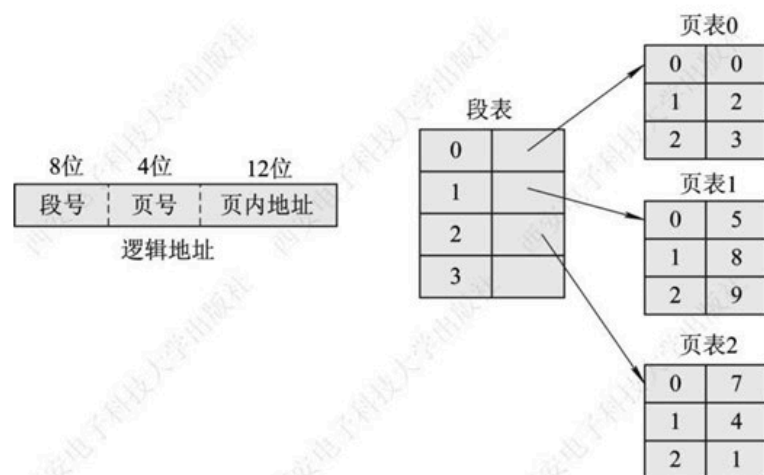
...

算法求工作集, 根据代码局部性分配页框, 动态调整页框个数, 页框确定后利用Clock算法换入换出



填空题 1分

在一个段页式系统中，某作业的段表、页表如图4-5所示，计算逻辑地址67392所对应的物理地址。 [填空1]



1. 将67392除以2，得到商33696和余数0。
2. 将商33696除以2，得到商16848和余数0。
3. 将商16848除以2，得到商8424和余数0。
4. 将商8424除以2，得到商4212和余数0。
5. 将商4212除以2，得到商2106和余数0。
6. 将商2106除以2，得到商1053和余数0。
7. 将商1053除以2，得到商526和余数1。
8. 将商526除以2，得到商263和余数0。
9. 将商263除以2，得到商131和余数1。
10. 将商131除以2，得到商65和余数1。
11. 将商65除以2，得到商32和余数1。
12. 将商32除以2，得到商16和余数0。
13. 将商16除以2，得到商8和余数0。
14. 将商8除以2，得到商4和余数0。
15. 将商4除以2，得到商2和余数0。
16. 将商2除以2，得到商1和余数0。
17. 将商1除以2，得到商0和余数1。

在段页式系统中，逻辑地址由段号、页号和页内地址组成。本题的页内地址为12位，所以页长为 2^{12} ，对应的物理块大小也为 2^{12} 。

69732 (**67392**) 对应的二进制数为10000011101000000 (模2取余数，倒序排列，前面补0 (7位))，分解为：

$$\underbrace{00000001}_{\text{段号 S}} \underbrace{0000}_{\text{页号 P}} \underbrace{0111010000}_{\text{页内地址 W}} \underbrace{00}_{\text{}}$$

所以段号S为1，页号P为0（查段号1对应的页表1中的表项0，知其对应的物理块为5），页内地址= $(011101000000)_2=1856$ 。故物理地址为 $5 \times 2^{12} + 1856 = 22336$ 。

主观题 10分

(考研真题) 请求分页管理系统中, 假设某进程的页表内容如表 1-6-5 所示。页面大小为 4KB, 一次内存的访问时间是 100ns, 一次快表的访问时间是 10ns, 处理一次缺页的平均时间是 108 ns (已含更新快表和页表的时间), 进程的驻留集大小固定为 2, 采用 LRU 页面置换算法和局部淘汰策略。假设: ①快表初始为空; ②地址转换时先访问快表, 若快表未命中, 则再访问页表 (忽略访问页表之后的快表更新时间); ③有效位为 0 表示页面不在内存中, 产生缺页中断, 缺页中断被处理后, 返回产生缺页中断的指令处重新执行。设有虚地址访问序列 2362H、1565H、25A5H, 请问:

(1) 依次访问上述 3 个虚地址, 各需要多少时间? 给出计算过程。

(2) 基于上述访问序列, 虚地址 1565H 的物理地址是多少? 请说明理由。

表1-6-5 进程页表内容

页号	页框号	有效位 (存在位)
0	101H	1
1	—	0
2	254H	1

(1) 页面大小为4KB=212B, 故页内偏移量为12位, 占虚地址的低12位, 高4位为页号。虚地址2362H的页号为2, 页内地址为362H。先访问快表(10ns), 未命中; 再去访问页表(100ns), 获得对应的物理块号254H, 其与页内地址362H拼接成物理地址254362H, 同时将第2页的信息装入快表中; 最后根据这个物理地址访问内存(100ns)。因此, 访问到虚地址对应单元的数据总共需要 $10\text{ns}+100\text{ns}+100\text{ns}=210\text{ns}$ 。

同理, 虚地址1565H的页号为1, 页内地址为565H。先访问快表(10ns), 未命中; 访问页表(100ns), 不在内存中, 产生缺页中断, 中断处理时间为108 ns, 然后返回产生缺页中断的指令处重新执行, 并以此更新快表和页表; 访问快表(10ns)便可得到对应的物理地址, 根据物理地址访问内存(100ns)。总共需要 $10\text{ns}+100\text{ns}+108\text{ ns}+10\text{ns}+100\text{ns}\approx 108\text{ ns}$ 。

同理, 虚地址25A5H的页号为2, 页内地址为5A5H。先访问快表(10ns), 该页信息已装入快表, 获得对应的物理块号254H, 从而拼接成物理地址2545A5H, 根据该地址访问数据(100ns)。总共需要110ns。

(2) 访问1565H时, 产生缺页中断, 在处理中断时, 合法驻留集为2, 内存已满, LRU页面置换算法将选择淘汰0号页, 然后将101H号页框分配给1号页, 这样便可得到页对应的物理块号101H, 其与页内地址565H拼接成的物理地址为101565H。

第四章（完）